



Testing Tactics

กลยุทธ์และเทคนิคการทดสอบซอฟต์แวร์เชิงลึก



รายชื่อสมาชิกกลุ่ม

1. อรปรียา อาสนทอง 6605001699
2. อภิญญา วงษ์ศรี 6605005344
3. กิมทีนา ลี 6705006069
4. สุทธิดา ทรงรัมย์ 6605005716
5. วิญญู สายแสง 6805007405

| Testing Tactics คืออะไร?

Testing Tactics คือกลยุทธ์หรือเทคนิคระดับรายละเอียดที่ใช้เลือกที่จะสร้าง Test Case อย่างไร เพื่อให้การทดสอบสามารถค้นหาข้อผิดพลาดได้อย่างมีประสิทธิภาพสูงสุด

แนวทางสำคัญคือการทดสอบผสมกันระหว่าง:

- **White-box Testing:** ตรวจสอบโครงสร้างภายในของโปรแกรม
- **Black-box Testing:** ตรวจสอบพฤติกรรมที่ผู้ใช้มองเห็น

ข้อควรจำ: การใช้เพียงแนวทางเดียวไม่เพียงพอ เนื่องจาก White-box และ Black-box มีจุดเน้นต่างกันและค้นพบ Bug คนละประเภท

| 1.1 พื้นฐานการทดสอบซอฟต์แวร์

ความหมายและเป้าหมาย

Software Testing: กระบวนการตรวจสอบด้วยการกำหนดข้อมูลนำเข้า เงื่อนไข หรือสถานการณ์ แล้วสังเกตผลลัพธ์เพื่อค้นหาข้อผิดพลาด

เป้าหมายสำคัญไม่ใช่เพื่อพิสูจน์ว่าซอฟต์แวร์ไม่มี Bug แต่เพื่อเพิ่มโอกาสค้นหา Error ที่มีอยู่

วัตถุประสงค์ของการทดสอบ

- ค้นหาข้อผิดพลาดและตรวจสอบการทำงานตามข้อกำหนด
- ตรวจสอบความถูกต้องของฟังก์ชันในกรณีปกติและผิดปกติ
- ตรวจสอบส่วนติดต่อระหว่างองค์ประกอบของระบบ
- เพิ่มความมั่นใจในคุณภาพภาพรวมของซอฟต์แวร์

| หลักการและคุณสมบัติซอฟต์แวร์ที่ทดสอบได้

หลักการสำคัญ

- เป้าหมายเพื่อค้นหาข้อผิดพลาด (Test Case ที่ดี มีโอกาสสูงในการเจอ Error)
- ไม่สร้าง Test Case ซ้ำซ้อน และให้ความสำคัญกับค่าขอบเขต (Boundary)
- ไม่สามารถทดสอบทุก Input ที่เป็นไปได้ จึงต้องเลือกอย่างมีหลักการ
- เริ่มทดสอบจากหน่วยเล็กไปใหญ่
(Component → Integration → System)

คุณสมบัติซอฟต์แวร์ที่ทดสอบได้ (Testable)

- **Operable / Observable / Controllable:** ทำงานดี สังเกตและควบคุมง่าย
- **Decomposable:** แบ่งย่อยเป็นส่วนๆ ได้
- **Simple / Stable / Understandable:** เรียบง่าย มั่นคง และเข้าใจโครงสร้างชัดเจน

1.2 การทดสอบกล่องขาว (White-box Testing)

เป็นการออกแบบ Test Case โดยพิจารณา โครงสร้าง ภายในของโปรแกรม เช่น Source Code, Control Flow, Conditions, Loops และ Data Structures

เป้าหมายหลัก:

- ให้ Statement ถูก execute อย่างน้อย 1 ครั้ง
- ให้ Logical Decision ถูกทดสอบทั้ง True และ False
- ตรวจสอบ Loop ที่บริเวณขอบเขตและเงื่อนไขต่างๆ
- ตรวจสอบโครงสร้างข้อมูลภายใน (Internal Data Structures)

```
if (age >= 18)
    allow();
else
    reject();
```

ตัวอย่าง: White-box สนใจว่า Test Case เดินผ่านครบทั้ง Branch True (age=20) และ False (age=15) หรือไม่ แทนที่จะมองแค่ผลลัพธ์ภายนอกอย่างเดียว

1.3 การทดสอบเส้นทางมูลฐาน (Basis Path Testing)

เสนอโดย **Tom McCabe** ใช้ Control Flow Graph เพื่อกำหนดเส้นทางอิสระ (Independent Paths) ผ่านโปรแกรม

สูตร Cyclomatic Complexity $V(G)$:

$$V(G) = E - N + 2 \quad (\text{เมื่อ } E=\text{Edges}, N=\text{Nodes})$$
$$\text{หรือ } V(G) = P + 1 \quad (\text{เมื่อ } P=\text{Predicate Nodes})$$

ค่าที่ได้ใช้กำหนดจำนวน Test Cases ขั้นต่ำเพื่อครอบคลุมเส้นทางพื้นฐาน

ข้อดีและข้อจำกัด

- **ข้อดี:** มีระบบระเบียบชัดเจน ช่วยค้นหา Logic Error เหมาะกับ Module สำคัญ
- **ข้อจำกัด:** ต้องเข้าใจ Code ภายในลึกซึ้ง และหากระบบซับซ้อนมาก จำนวน Path จะเพิ่มขึ้นสูง

| 1.4 การทดสอบโครงสร้างควบคุม (Control Structure)



1. Condition Testing

ตรวจสอบเงื่อนไขในคำสั่ง เช่น
if (A && B) เพื่อทดสอบค่า
True/False ค้นหา Error ใน
Logical Operators (&, |, !)



2. Data Flow Testing

ติดตามการใช้ตัวแปรตั้งแต่
Definition จนถึง Use ค้นหา
ปัญหาใช้ตัวแปรยังไม่กำหนด
ค่า หรือกำหนดแล้วไม่ได้ใช้



3. Loop Testing

ทดสอบรอบของ Loop หลาย
กรณี เช่น 0, 1, 2, ปกติ, สูงสุด
และเกินขอบเขต (Boundary)

| 1.5 การทดสอบกล่องดำ (Black-box Testing)

เป็นการออกแบบ Test Case โดย ไม่พิจารณาโครงสร้างภายใน แต่ดูจาก Functional Requirements, Specification และ Expected Output (Specification-based)

สิ่งที่มุ่งตรวจสอบ: ฟังก์ชันผิดพลาด, ปัญหา Interface, การเข้าถึงฐานข้อมูล, พฤติกรรมและ Performance

เทคนิคสำคัญของ Black-box

- **Equivalence Partitioning:** แบ่งกลุ่มข้อมูลนำเข้าที่มีพฤติกรรมเหมือนกัน แล้วเลือกตัวแทนมาทดสอบ
- **Boundary Value Analysis:** เน้นทดสอบค่าบริเวณขอบเขตซึ่งมักเกิด Error บ่อยที่สุด
- **Graph & Model-based Testing:** สร้างกราฟหรือแบบจำลองสถานะเพื่อออกแบบ Test Case

| เปรียบเทียบ White-box และ Black-box

ประเด็นเปรียบเทียบ	White-box Testing	Black-box Testing
สิ่งที่สนใจ	โครงสร้างภายใน (Source Code / Flow)	พฤติกรรมภายนอก / ข้อกำหนด (Spec)
การเห็น Source Code	จำเป็นต้องรู้โครงสร้างภายใน	ไม่จำเป็นต้องรู้โครงสร้างภายใน
ตัวอย่างเทคนิค	Basis Path, Condition, Data Flow, Loop	Equivalence Partitioning, Boundary Value
จุดเด่น	ตรวจสอบ Logic และ Control Flow	ตรวจสอบ Function ที่ผู้ใช้เห็นจริง

| 1.6 วิธีทดสอบเชิงวัตถุ (OO Testing)

Object-Oriented Testing ต้องคำนึงถึงคุณสมบัติพิเศษ เช่น Class, Object, Encapsulation, Inheritance, Polymorphism และ Message Passing

หน่วยย่อยที่เหมาะสมสำหรับการทดสอบคือ Class/Object ที่ห่อหุ้ม State และ Operations ไว้ด้วยกัน

ประเด็นความแตกต่างของ OO

- **Encapsulation:** ข้อมูลภายในถูกซ่อน ต้องทดสอบผ่าน Public Interface
- **Inheritance:** ต้องเช็คพฤติกรรม Subclass เมื่อรับทอดจาก Superclass
- **Polymorphism:** Method เดียวกันอาจทำงานต่างกันตามชนิด Object
- **State:** ผลลัพธ์เปลี่ยนไปตาม State ปัจจุบันของ Object

1.7 การทดสอบ ณ ระดับคลาส

1. Random Testing for OO Classes

สร้างลำดับการเรียก Operation แบบสุ่มแต่ต้องสอดคล้องกับข้อจำกัดของ State (เช่น ห้าม withdraw ก่อน open)

```
open → setup → deposit → balance → withdraw → close
```

ช่วยค้นหาพฤติกรรมจากลำดับการเรียก Method ที่หลากหลาย

2. Partition Testing at Class Level

แบ่งกลุ่ม Test Case เพื่อลดจำนวนรูปแบบแต่ยังคงครอบคลุมพฤติกรรมสำคัญ:

- **State-based:** แบ่งตามสถานะของ Object (New, Active, Closed)
- **Attribute-based:** แบ่งตามค่าหรือช่วงของ Attribute
- **Category-based:** แบ่งตามประเภทข้อมูลหรือพฤติกรรม

| 1.8 การออกแบบกรณีการทดสอบระหว่างคลาส

เป็นการทดสอบ การทำงานร่วมกันและการสื่อสารระหว่างหลาย Class แม้แต่ละ Class จะผ่าน Unit Test แล้ว แต่เมื่อทำงานร่วมกันอาจเกิดปัญหา เช่น ส่ง Parameter ผิด, เรียก Method ผิดลำดับ หรือ State ไม่ตรงกัน

แนวทาง Multiple Class Testing: ระบุ Client และ Server Class, กำหนด Sequence Operation, และตรวจสอบผลลัพธ์รวมของ Collaboration

Tests Derived from Behavior Models

สร้าง Test Case จาก Behavioral Model เช่น State Diagram หรือ Sequence Diagram

Created → Paid → Preparing → Completed

ตรวจสอบทั้ง Transition ที่ถูกต้องและป้องกัน Transition ที่ห้ามเกิดเด็ดขาด

Q & A

แหล่งอ้างอิง:

- Roger Pressman, "Software Engineering: A Practitioner's Approach", 8th edition
- เอกสารประกอบการเรียนและแหล่งข้อมูลวิชาการด้าน Software Testing